ΔELTA

# Guide to Building Your First Chatbot

with Python, Visual Studio Code and an API key

## Introduction

Welcome to a beginner-friendly guide designed to walk you through creating your very first chatbot. We will use Python, Visual Studio Code, and an API key to create a simple AI-based web interface for interaction. Fear not if you're completely new to coding, every step of this journey is designed to be beginner-friendly, taking you from novice to chatbot creator in just a few hours.

## Prerequisites

Absolutely none! This guide has been designed with complete beginners in mind, and we'll explain every step in detail. All you need is a computer, internet connection, and a thirst to learn!

## Table of Content


1. **Understanding the Basics:** A brief overview of the tools, systems and platforms we will use to power our chatbot.
2. **Setting Things Up:** Detailed steps for installing Python, Visual Studio Code and accessing the relevant API keys.
3. **Building the Chatbot:** Line-by-line guide for coding your chatbot. Each code snippet will be explained, so you understand what you're doing.
4. **Creating a Web Interface:** A beginner-friendly guide to creating a simple web interface using HTML, allowing easy interaction with your chatbot.
5. **Wrap Up and Next Steps:** A review of what you've learned and achieved, with suggestions for ways to further improve your chatbot and skills.


## Appendix


**How to get the Most out of Coding with a Chatbot:** Best Practices on how to code with Chatbots.
**Glossary of Technical Terms:** A quick reference that explains technical terms and jargon in simple language.
**Common Python Errors:** A summary of the most common Python errors.

# 1. Understanding the Basics

In this section, we'll introduce you to the fundamental concepts and tools that you'll be using to create your chatbot. Here's what we'll cover:

- Python—the main programming language for this project
- Visual Studio Code—a tool for writing and running code
- Large Language Models (LLMs)—the backbone of our chatbot
- The LLM API—the engine that will connect our chatbot with an LLM

By the end of this section, you'll have a foundational understanding of the tools and concepts needed to build your chatbot.

## Introduction to Python

Python is a high-level, interpreted programming language, named not after the snake, but after the British comedy troupe Monty Python! Since its creation in the late 1980s by Guido van Rossum, Python has gained immense popularity. As of now, it has millions of users worldwide and has been used to write billions of lines of code.
Here's why Python is such a hit, especially in fields like machine learning and artificial intelligence:

- **Readability:** Python's syntax is clean and intuitive. It's designed to be easy to read and write, making it an excellent choice for beginners.
- **Breadth of Libraries:** Python boasts a wide array of libraries—pre-written code that can perform specific tasks or operations. Think of them as toolkits that expedite your coding process. For our chatbot, we'll tap into the Gemini library.
- **Community Support:** Python enjoys one of the largest communities of developers on the planet, making it easy to find help, tutorials and advice.
- **Versatility:** Python is a jack-of-all-trades. Whether it's web development, data analysis, game development, or AI, Python has you covered.
- **Interactivity:** Python's interactive mode makes debugging easy and straightforward. You can test out a chunk of code anytime you want.

Be sure to also check out `Introduction to Python for AI.ipynb`, which offers a practical introduction to using Python.

## Introduction to Visual Studio Code

Visual Studio Code (VS Code) is one of the most popular code editors in the world, created and maintained by Microsoft. Let's look at some of the reasons why millions of developers, including us, choose to write their code in VS Code:

- **Versatility and Power:** VS Code supports almost all programming languages, from JavaScript to Python. Just install the language you want to use and you are ready to start coding.
- **Built-in Terminal:** VS Code comes with an integrated terminal, making it easy to run your Python scripts without having to leave the editor.
- **Extensibility**: In the mood for more? VS Code's marketplace offers thousands of extensions to enrich your coding experience. Be sure to try out the Python extension for some nifty features!
- **Git Integration**: VS Code integrates seamlessly with Git and other version control systems, enabling you to effortlessly track changes and synchronize with remote repositories.

Here's a fun fact: Not only is VS Code popular and powerful, but it's also open source - anyone can contribute to its development. Check out VS Code's GitHub repository to peek under the hood!

## A Primer on Large Language Models (LLMs)

Large Language Models (LLMs) are a form of Artificial Intelligence that's trained to understand and generate human language. Here's what you should know about LLMs:

- **Data Training:** LLMs are trained on a diet of vast amounts of text data, from books and articles to web pages. They learn to predict what word or phrase is likely to come next, based on the context.
- **Flexibility:** After training, LLMs can perform a wide range of language tasks, from answering questions to writing essays, translating languages, and even crafting poetry.
- **Chatbot Applications:** Some of the most advanced chatbots, like Gemini's ChatGPT, are powered by LLMs. For our chatbot, we'll use an LLM provided by Gemini to help our chatbot understand and respond to user inputs.

LLMs represent an exciting milestone in the evolution of AI. They've moved us beyond simple rule-based systems to models that can work with unstructured data and can generate human-like text.

## Understanding LLM APIs

Before we delve into LLM APIs specifically, let's understand what an API, or Application Programming Interface, is. In simple terms, an API is a set of rules that allow different software applications to communicate with each other. It's like a menu in a restaurant - you (the customer) can see a list of dishes you can order

(the services), but the kitchen (the system you're accessing) handles the details of how to prepare them.
Now, let's talk about LLM APIs. These give you access to a wide range of base LLMs which would be much too expensive to train and run on your own. Here's what you need to know about them:

- **Functionality:** LLM APIs allow us to harness the power of LLMs from many different providers. We will use them to get responses to user inputs for our chatbot.
- **Ease of Use:** LLM APIs are designed to be straightforward and easy to use. We'll send it some text (the user's input), and it sends us back some generated text (the chatbot's response).
- **Potential:** LLM APIs have broad applications. While we're focusing on building a chatbot now, the same API could be used to create tools for writing assistance, tutoring, translation, and so much more.

# 2. Setting things up

In this section we will go through the steps for downloading and installing all the relevant tools to get started with our project. Specifically, you will learn how to:

- Install Python
- Install Visual Studio Code
- Get your own LLM API key

By the end of this chapter, you'll be equipped with all the necessary tools and knowledge to dive head-first into coding!

## Installing Python

**Download Python:** Navigate to the official Python website (www.python.org). Select the 'Downloads' tab and click on the version recommended for your operating system to download the installer.

**Run the Installer:** After the download, locate the Python installer file in your Downloads folder. The file should be named something like 'python-3.x.x.exe', where '3.x.x' is the version number. Double-click this file to start the installation process.

**Install Python:** In the Python Setup window, make sure to check the box that says 'Add Python 3.x.x to PATH'(if prompted), then click 'Install Now'. Wait for the installation to complete.

**Verify the Installation:** To confirm Python was installed correctly, open a new command line window ('Command Prompt' on Windows, 'Terminal' on macOS or Linux) and type 'python --version'. If the installation was successful, it should display the Python version you installed.

## Installing Visual Studio Code

**Download VS Code:** Head over to the Visual Studio Code official website (https://code.visualstudio.com/). Click the big blue 'Download' button, which will automatically detect your operating system and download the correct version for you.

**Install Visual Studio Code:** Once the download is complete, locate the Visual Studio Code installer in your Downloads folder. Follow the prompts in the Setup wizard. Accept the license agreement and leave the default settings as they are. Click 'Install' and wait for the installation to complete.

# Obtaining an LLM API Key

For your chatbot to function it needs to be connected to an underlying base LLM, which facilitates intelligent response from the bot. Access to these LLMs is typically provided through APIs, where users have to pay per use.

To manage user accounts and their billing information, API providers issue a secret key that connects the user's identity with their API usage. There are multiple API providers and each has slightly different ways of issuing keys. Deltaschool will provide you with your API key via Discord. Be sure to keep it safe and to never share it with anyone!

As an example for future projects, we will show you how to receive an API key from Gemini as a private user as well:

**Sign Up for Gemini:** Head over to Gemini's official website
https://ai.google.dev/gemini-api/docs/api-key
and click 'Get an API key' to register your key.

Click to copy the key under ‘API key’ section. This is your API key for Gemini model.

Click the sign in front of ‘Generative Language Client’ under ‘Project ID’. Then click on the key name under ‘API Keys’ and ensure under ‘API restrictions’ you select ‘Don't restrict key’ option.

# 3. Building the Chatbot

In this section we will finally get our hands dirty and write our first lines of code to bring our chatbot to life! We will break things down into three steps:

- Configuring the coding environment in Visual Studio Code
- Building the Chat Function, which sends requests and receives responses from the Gemini API
- Interacting with the Chatbot via the terminal

By the end of this chapter, you will have successfully developed a chatbot that can be accessed and interacted with through your computer's command line interface.

## Configuring the Coding Environment

Before we dive in, let's quickly define what we mean by a 'coding environment'. A coding environment is essentially the setup on your computer where you write, test, and run your code. It includes the text editor or integrated development environment (IDE) you use to write your code (in our case, Visual Studio Code), the programming language you're using (Python), and any libraries or packages your code will use.
Now, let's set up our coding environment:

**Create a Project Folder in VS Code:** Launch Visual Studio Code. Click 'File' from the menu bar at the top, then select 'Open Folder'. In the dialog that opens, navigate to where you would like to place your folder for this project (a common location is ‘Documents’) and create a new folder named 'YourCodingChatbot'. This is your project workspace, where you'll write and store your chatbot code.

**Generate a New Python File**: Go to the menu bar and click on 'File' -> 'New File'. This will open a new, blank file. Save this file in your project folder with a .py extension, which denotes a Python file. We will name ours ‘chatbot.py’.

**Install the Gemini API Library:** We need to install the Gemini API library in order to call the Gemini API. Open the terminal within VS Code by going to 'View' -> 'Terminal'. Then select the terminal and type the following command and press enter:

```
pip install -U google-generativeai
```

This command uses pip, Python's package installer, to install the Gemini package. You should see output in your terminal that suggests Gemini has been successfully installed.

**Import the Gemini API Library:** At the top of your chatbot.py file, type:
**import google.generativeai as genai**

This line of code tells Python that we're going to use the Gemini library in our program.

**Set up the API Key**: Now, let's set up our Gemini API key. Below the 'import Gemini' line, add:
**genai.configure(api_key='your-api-key')**

Replace 'your-api-key' with the key you received when you created your Gemini account (see Section 1). Be careful not to share this key, as it's linked to your account!

With these steps, you've successfully set up your coding environment! In the next section, we'll dive into writing the function that powers the chatbot.

## Building the Chat Function

In this section, we will be writing a Python function that takes a user's input and gets a response from the Gemini API.

Our get_response function should look like this:

```
# Initialize the model
model = genai.GenerativeModel('gemini-1.0-pro')
# Define a function to get responses from the model
def get_response(message):
  # Generate the response
  response = model.generate_content(message)
  # Extract and return the response text
  return response.text

# Example usage
user_message = "Hello, how are you?"
response = get_response(user_message)
print(response)
```

And with that, we have our chat function! In the next section, we'll see how to run our chatbot and see it in action.

## Interacting with the Chatbot

Now that we have our chat function ready, let's write some code that will let us talk with our chatbot through a simple text-based interface in the terminal.

**Start a Chat Loop:** Below your get_response function, let's start an infinite loop. This loop will keep running until we explicitly decide to stop it:

```
while True:
```

**Get the User's Message**: Inside the loop, the first thing we need to do is get a message from the user. We can do this with the input() function, which reads a line of text that the user types into the terminal. Let's prompt the user to type their message with a simple instruction:

```
user_message = input("You: ")
```

**Get the Chatbot's Response:** Next, we'll call our get_response function to get a response from the chatbot. We'll pass the user's message to the function, like so:

```
chatbot_response = get_response(user_message)
```

**Print the chatbot's response:** Finally, let's print out the chatbot's response. We'll prepend it with "Chatbot: " to make it clear who's speaking:

```
print("Chatbot: ", chatbot_response)
```

Putting it all together and adding a print statement at the end to display the output, the code for interacting with the chatbot should look like this:

```python
import google.generativeai as genai
# Configure the API key
genai.configure(api_key='your-API-key')


# Initialize the model
model = genai.GenerativeModel('gemini-1.0-pro')
# Define a function to get responses from the model
def get_response(message):
  # Generate the response
  response = model.generate_content(message)
  # Extract and return the response text
  return response.text

# Example usage
while True:
  user_message = input("You: ")
  chatbot_response = get_response(user_message)
  print("Chatbot: ", chatbot_response)
```

And that's it!

If you run your chatbot.py file now (you can do this by clicking on ' Run' -> 'Run Without Debugging' in the menu bar), you should be able to have a conversation with your chatbot right in the terminal!

After running this command, you should see that the terminal is waiting for your input. You can start a conversation by typing a message, like so:

```
You: Hi there!
```

Once you hit 'Enter', the chatbot should respond. For example, it might say something like:

```
Chatbot: Hello! How can I assist you today?
```

Remember, this is an infinite loop, so you can keep chatting with your bot as long as you want! To stop the conversation, you can simply use the keyboard shortcut Ctrl + C in the terminal.

# 4.Creating a Web Interface

In this section, we'll create a web interface for our chatbot and link it to our Python chatbot code. This section will cover the following topics:

- Creating a chat interface using HTML
- Creating a Flask app
- Connecting our chatbot with the chat interface
- Interacting with our enhanced chatbot

## Creating a Chat Interface

HTML (Hyper Text Markup Language) is the standard language for creating web pages. It uses 'tags' to structure and style content. In this section, we'll create a basic chat interface using HTML.

**Create an HTML file:** In your project directory, create a new file called 'chatbot.html'.

**Basic HTML Structure:** Open 'chatbot.html' in Visual Studio Code and add the following code:

```
<!DOCTYPE html>
<html>
<head>
<title>My Chatbot</title>
</head>
<body>
<!-- Chatbot interface will go here -->
</body>
</html>
```

This is the basic structure of any HTML document. The <head> tag contains the title of the page, and the <body> tag is where we'll add our chat interface.

**Chat Display Area:** Now, we're going to create a place to display the chat messages. Add the following code inside the <body> tag:

```
<div id="chatArea">
</div>
```

Here,  is an HTML tag that creates a division or a section in an HTML document. We're using it to create a dedicated space for the chat messages. The

‘id’ attribute is used to give the tag a unique identifier, which we'll use later to display chat messages.

**User Input Field:** Next, let's create an area where users can enter their messages. We'll use the <input> tag for this. Add the following code inside the <body> tag, just below the chatArea div:

```
<input type="text" id="userInput" placeholder="Type a message...">
```

The ‘placeholder’ attribute is used to provide a hint that describes the expected value of an input field. That's it! You've created a simple chat interface.

**Opening the HTML File:** To see your chat interface, you need to open the chatbot.html file in a web browser. Here's how you can do it:

1. In Visual Studio Code, right-click on the chatbot.html file in the Explorer panel on the left.
2. Click on 'Reveal in File Explorer' in Windows or 'Reveal in Finder' for macOS).
3. This will open the location of your chatbot.html file in your system's file manager.
4. Double-click the chatbot.html file to open it in your default web browser.

You should see an input field where you can type a message. However, if you type a message and hit 'Enter', nothing will happen. We'll handle user inputs and display chat messages in the following sections.

In the next section, we'll introduce Flask, a Python framework that will help us link our Python chatbot to this HTML interface.

## Creating a Flask App

Flask is a simple library which helps us connect our Python code to a website.

**Delete the Infinite Loop:** As we are going to connect our python program to a web interface, we don’t need the loop that we created previously, so let’s delete it.

**Install Flask:** Open your terminal and run the following command to install Flask:

```
pip install flask
```

**Import Flask:** At the top of ‘chatbot.py’, add the following line to import Flask:

```
from flask import Flask, render_template, request
```

**Create a Flask App:** Add the following lines to 'chatbot.py' below your get_response function:

```
app = Flask(__name__)

@app.route('/')
def home():
  return render_template('chatbot.html')
```

With these lines, when we go to the webpage at the address 'http://localhost:5000/', Flask will load our 'chatbot.html'. 'The chatbot.py' file should now look like this:

```
from flask import Flask, render_template, request
import google.generativeai as genai

# Configure the API key
genai.configure(api_key='your-API-key')


# Initialize the model
model = genai.GenerativeModel('gemini-1.0-pro')
# Define a function to get responses from the model
def get_response(message):
  # Generate the response
  response = model.generate_content(message)
  # Extract and return the response text
  return response.text

app = Flask(__name__)

@app.route('/')
def home():
  return render_template('chatbot.html')
```

In the next section, we'll handle the user inputs from our chat interface and connect our chatbot logic.

## Connecting the Chat Interface with Your Chatbot

Now, we are going to connect the chat interface with the chatbot. We'll need to update our two files: ‘chatbot.py’ and ‘chatbot.html’.

**Update ‘chatbot.py’:** We'll create a new path that the chat interface can send user messages to, and get responses from our chatbot. Update the ‘chatbot.py’ by adding the following code at the end of the file:

```
@app.route('/ask', methods =['POST'])
def ask():
  message = request.form['message']
  response = get_response(message)
  return response

if __name__ == "__main__":
  app.run()
```

**Update chatbot.html:** Let's update ‘chatbot.html’ to send the user's input to our server and show the chatbot's response (We will also add some additional formatting for displaying code nicely). Add the following script at the end of the ‘chatbot.html’ file:

```
<script
 src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"><
/script>
<script>
$(document).ready(function() {
    $('#userInput').on('keypress', function(e) {
        if (e.which == 13) { //Detect enter key
e.preventDefault(); //Prevent form submission var message
            = $(this).val();
$.post("/ask", {message: message}, function(data) {
//Add user's message to the chat area
$('<p>').text('You: ' + message).appendTo('#chatArea');

// Split the chatbot's response into blocks var blocks =
                data.split('```');
for (var i = 0; i < blocks.length; i++) {
// Check if the block is code
if (blocks[i].startsWith('python')) {
// Remove 'python' and wrap in <pre><code> tags var code =
                        blocks[i].slice(6);

$('<pre>').append($('<code>').text(code)).appendTo('#chatArea');
```

```
} else {
// Split the block into lines and add each line
to the chat area
var lines = blocks[i].split('\n');
for (var j = 0; j < lines.length; j++) {

$('<p>').text(lines[j]).appendTo('#chatArea');
}
}
}
});
$(this).val(''); //Clear input field
}
});
});
```

So your final files should look like this:

**chatbot.py:**

```python
from flask import Flask, render_template, request
import google.generativeai as genai

# Configure the API key
genai.configure(api_key='your-API-key')


# Initialize the model
model = genai.GenerativeModel('gemini-1.0-pro')
# Define a function to get responses from the model
def get_response(message):
  # Generate the response
  response = model.generate_content(message)
  # Extract and return the response text
  return response.text

app = Flask(__name__)

@app.route('/')
def home():
  return render_template('chatbot.html')


@app.route('/ask', methods =['POST'])
def ask():
  message = request.form['message']
  response = get_response(message)
  return response

if __name__ == "__main__":
  app.run()
```

**chatbot.html:**

```
<!DOCTYPE html>
<html>
<head>
<title>My Chatbot</title>
</head>
<body>
<div id="chatArea">
</div>
<input type="text" id="userInput" placeholder="Type a message...">
</body>
</html>

<script
 src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"><
/script>
<script>
$(document).ready(function() {
    $('#userInput').on('keypress', function(e) {
        if (e.which == 13) { //Detect enter key
e.preventDefault(); //Prevent form submission var message
            = $(this).val();
$.post("/ask", {message: message}, function(data) {
//Add user's message to the chat area
$('<p>').text('You: ' + message).appendTo('#chatArea');

// Split the chatbot's response into blocks var blocks =
                data.split('```');
for (var i = 0; i < blocks.length; i++) {
// Check if the block is code
if (blocks[i].startsWith('python')) {
// Remove 'python' and wrap in <pre><code> tags
```

```
var code = blocks[i].slice(6);

$('<pre>').append($('<code>').text(code)).appendTo('#chatArea');
} else {
// Split the block into lines and add each line
to the chat area
var lines = blocks[i].split('\n');
for (var j = 0; j < lines.length; j++) {

$('<p>').text(lines[j]).appendTo('#chatArea');
}
}
}
});
$(this).val(''); //Clear input field
}
});
});
<script>
```

And there you have it! You now have a working chat interface for your chatbot. Test it out by opening your chatbot.html in a web browser and typing a message!

## Interacting with Your Chatbot

Now, it's time to bring your chatbot to life! Here are the steps:

**Run the Server:** Go back to 'chatbot.py' in Visual Studio Code. Run the file by clicking on 'Run' -> 'Run Without Debugging' in the menu bar.

**Open the Chat Interface**: The terminal should now display a webpage on which your program is running. Select it or copy it into the address bar and press Enter. You'll see your chat interface.

**Start Chatting:** Type your message into the input field and press Enter to see your chatbot's response!

Remember, keep the server running in Visual Studio Code while you're chatting with your chatbot.

And that's it! You've created a chatbot with a simple web interface. Have fun chatting with your new creation.

# 5. Wrap up and next steps

Congratulations! You've built your very own chatbot from scratch. You've not only learned how to code a basic chatbot using Python, but you've also seen how you can connect your Python application with a web interface.

## What You've Achieved

Here's a quick recap of what you've accomplished:

- You've installed and set up a coding environment (Python and Visual Studio Code).
- You've learned about Gemini, APIs, and how to interact with them using Python.
- You've coded a Python application that communicates with the Gemini API.
- You've created a web interface using HTML and Flask, and connected it to your Python application.

## Next Steps for Advancing Your Chatbot

Now that you've gotten a taste of coding a chatbot, the sky’s the limit! Here are a few suggestions for how you can take your project to the next level:

- **Add Streaming Functionality:** Your chatbot currently waits until it gets a complete response from Gemini before sharing the output with a user. Instead, you can modify it so that it streams every word that has been generated in real-time. This makes a big difference in user-experience as you can start reading the answer as it is being created.
- **Enhance the Interface:** The interface for your chatbot is quite basic. With knowledge of CSS and JavaScript, you could significantly improve the aesthetics and user experience. Consider learning these languages and applying them to your project.
- **Experiment with Different LLM Models:** The Gemini API offers different models to choose from. Each model has different strengths and weaknesses, and may perform better or worse depending on the type of conversation. Try experimenting with different models and see how they affect your chatbot's responses.
- **Add a Sentiment Analysis Component:** You could integrate a sentiment analysis module. This would allow your chatbot to gauge the sentiment behind user inputs and potentially adjust responses based on whether the user seems happy, sad, angry, etc.
- **Implement Error Handling and Testing:** As your chatbot grows more complex, it's important to make sure it's robust and can handle unexpected situations gracefully. Consider learning more about error

handling in Python, and implement comprehensive testing for your chatbot to ensure its quality.

- **Localization:** If you speak another language, try to localize your chatbot to understand and speak in that language. This not only expands your user base but also gives you an insight into how language processing works in different languages.
- **User Personalization:** You could also add features to your chatbot that allow it to remember specific users and customize its responses based on previous interactions with those users.

Remember, the path to mastery is through continuous improvement and experimentation. Your chatbot project provides a fantastic platform for you to learn and grow your coding skills. Happy coding!

# Appendix

## How to get the Most out of Coding with a Chatbot

**Chatbots as Personal Coding Tutors**
Consider the chatbot as a tutor who, while occasionally slow to grasp your queries, is infinitely patient. Unashamedly ask follow-up questions if you don't fully understand something. Regardless of how many times it takes, the chatbot will never tire of explaining concepts anew. The more explicit your questions, the more precise the responses. Always strive to give the chatbot as much context as possible.
There will be times when the chatbot provides incorrect answers or code. This is normal and nothing to worry about. Feel free to tell it. Specify what's wrong, and the chatbot will attempt to correct it. Each correction is a step towards more accurate results.

**What types of coding can Chatbots do well?**
The proficiency of a given chatbot in a given programming language depends on how much of the language was in its training data.
Generally, chatbots are proficient in common programming languages such as Python and JavaScript, owing to the abundant training data on these languages. However, they might not perform as well with more obscure languages or those where most of the potential training data is private, such as ServiceNow applications or Stata. But feel free to try sending it some code and you will soon get a grasp of the chatbot’s proficiency.

**Code Optimization**
Chatbots are reliable assistants in optimizing your code. They can ensure your code adheres to accepted standards, like Python's PEP 8 rules, and can also suggest ways to make your code more efficient and modular. They’re adept at suggesting error handling and logging practices, and can guide you in documenting your code effectively. These tasks are typically tedious, but chatbots excel at them!

**Understanding Context Windows**
Every chatbot has a certain memory limit called the 'context window'. On the main app of ChatGPT, it's 4k tokens, and with plugins, it extends to 8k tokens. Anything beyond these limits might get overlooked. When asking questions, especially unrelated ones, consider starting separate chats to ensure the chatbot has only the most relevant context to provide accurate responses.

**Break Boundaries and Experiment**
Coding with chatbots is an adventure. It's a platform to experiment, make mistakes, learn, and grow. Sometimes, you might be disappointed, but more

often, you'll be fascinated by the outputs. Don't hesitate to push boundaries. The more you interact, the more you'll learn, not just about coding, but about the mechanics of AI chatbots too.

## Glossary of Technical Terms

**API (Application Programming Interface):** A set of rules and protocols for building and interacting with software applications. It allows different software programs to communicate with each other.
**COMMAND LINE:** A text-based interface used to interact with your operating system.
**CSS (Cascading Style Sheets):** A stylesheet language used for describing the look and formatting of a document written in HTML. It's used to control the layout of multiple web pages all at once.
**EDITOR:** A program where you can write and modify your code.
**GIT:** A distributed version control system for tracking changes in source code during software development. It's designed for coordinating work among programmers.
**HTML (HyperText Markup Language):** The standard markup language for documents designed to be displayed in a web browser. It can be assisted by technologies such as CSS and JavaScript to create web pages.
**JAVASCRIPT:** A programming language that is primarily used to enhance web pages to make them more interactive.
**LARGE LANGUAGE MODEL:** A type of artificial intelligence model that can generate human-like text based on the input it is given.
**LIBRARY:** A set of code that other people have written to be easy for other programmers to do common tasks.
**LIST:** In Python, a list is a data structure that holds an ordered collection of items i.e. you can store a sequence of items in a list.
**LOOP:** A programming structure that repeats a sequence of instructions until a specific condition is met.
**MARKUP:** Refers to the notation or coding added to the content of a document that provides the web browser with the information about the structure and visual presentation of the text and images.
**PACKAGE:** In Python, a package is a way of organizing related modules into a single directory hierarchy.
**PYTHON:** A high-level, interpreted programming language known for its easy readability with great design principles.
**SCRIPT:** A piece of code written for a special run-time environment that automates the execution of tasks.
**STATEMENT:** In programming, a statement is a single line of code that performs a specific action.
**SYNTAX:** The rules of how to write code in a specific programming language. **TERMINAL:** Another name for the command line. It's a program that receives commands from the user and gives them to the operating system to perform.

**VARIABLE:** A name that is used to denote something or a value is called a variable. In python, variables can be declared and values can be assigned to it as per our requirements.
**VSCODE (Visual Studio Code):** A popular code editor. It's like a fancy word processor for writing code. It's free and available for Windows, macOS and Linux.

## Common Python Errors

When you're learning to code, running into errors is part of the process. Here are some common Python errors that you might encounter:

**SyntaxError:** Python can't understand your code. This often happens when you forget a colon (:) at the end of a statement, misspell a keyword, or forget to close a bracket. Always check the line number given in the error message, and the line of code above it.
**IndentationError:** Python uses indentation to define blocks of code. Make sure you are using consistent indentation - either tabs or spaces, not both.
**NameError:** You are trying to use a variable or function that hasn't been defined yet. Make sure all your variables and functions are spelled correctly and defined before you try to use them.
**TypeError:** You're using an operation or function with the wrong type of object. For example, trying to add a number to a string.

Remember, errors are your friends! They're here to tell you what's wrong with your code. When you encounter an error, take a deep breath, read the error message, and try to understand what it's telling you. And if you can't figure it out, don't hesitate to ask for help in the Discord channel!